

树套树

——成都七中 nzhtl1477

偏序维护

- 即每次对满足多维的一个限制的所有数进行操作
- 多维的限制：每个点 i 有 $a_i, b_i, c_i \dots$ 等不同的属性
- 每次对 $l_1 \leq a_i \leq r_1, l_2 \leq b_i \leq r_2 \dots$ 的 i 进行一次修改操作

如何维护？

- 1.Range Tree
- 2.K-D Tree
- 3.Quad Tree , Octree
- 4.R-Tree

Range Tree

- 其实就是狭义上的树套树

树套树

- 能在 $O(\log n^d)$ 的复杂度内进行一次 d 维偏序的区间查询
- 能在 $O(\log n^d)$ 的复杂度内进行一次 d 维偏序的单点修改
- 空间为 $O(n \log n^{(d-1)})$, 可以优化到 $O(n(\log n / \log \log n)^{(d-1)})$, 不过这个应该没人会所以可以无视掉

树套树

- 具体来说什么树套什么树是有关系的

树套树

- 如果要维护 d 维, 出于方便设每维的值大小是 v 的一个偏序

高维树状数组

- 本质就是树状数组的嵌套
- 时间复杂度 $O(\log v^d)$, 空间复杂度 $O(v^d)$
- 可以通过预先高维离散化来做到
- 时间复杂度 $O(\log v^d)$, 空间复杂度 $O(n \log v^d)$
- 不过这个基本上无意义

~

- 接下来只讨论二维情况
- 因为三维情况下常数和空间都起飞了， 就没见过一个三维的题

关于树套树

- 我最开始也觉得这东西很牛逼，很码农
- （后面发现也就5分钟的事。。。）
- 感觉能用到的树套树基本上都可以用树状数组套平衡树/线段树来写

函数化数据结构的思想

- 我们以树状数组套平衡树作为例子来介绍一下树套树

普通的树状数组

- 维护一个序列支持:
- 1.把x位置的值加上y
- 2.查询一个区间的和

```
inline void modify( int x , int y )
{
    for( register int i = x ; i <= n ; i += lowbit( i ) )
        t[i] += y;
}

inline int find( int x )
{
    int ans = 0;
    for( register int i = x ; i ; i -= lowbit( i ) )
        ans += t[i];
    return ans;
}
```

树状数组套平衡树

- 维护一个序列支持:
- 1.把x位置的值改为y
- 2.查询一个区间中小于y的数个数

```
inline void insert( int x , int y )
{
    for( register int i = x ; i <= m ; i += lowbit( i ) )
        insert( y , root[i] );
}

inline int find( int x , int y )
{
    int ans = 0;
    for( register int i = x ; i ; i -= lowbit( i ) )
        ans += rank( y + 1 , root[i] );
    return ans;
}
```

区别

- 普通树状数组用到的数据结构：支持修改值，查询值——变量
- 所以普通的树状数组可以用一个数组来维护
- 树状数组套平衡树用到的数据结构：支持插入一个值，查询小于一个值的数个数——平衡树
- 所以树状数组套平衡树可以用一个平衡树的数组维护

区别

- 变量每次访问是 $O(1)$ 的，所以树状数组时间复杂度是 $O(\log n)$
- 平衡树每次访问是 $O(\log \text{size})$ 的，所以树状数组套平衡树的最坏时间复杂度是 $O(\log^2 n)$ 的
- 由于不是每棵平衡树都满，所以这里带一个小常数，然而平衡树常数比较大，所以还是比较慢的

所以

- 只需要先写一个平衡树，然后写个树状数组就行了

```

struct Node
{
    int size , value;
    Node * left , * right;
    Node( int s , int v , Node * a , Node * b ) : size( s ) , value( v ) , left( a ) , right( b ) {}
    Node() {}
} * null , ████████████████████ , * root[ MAXN ];

int n , m , cnt;

inline void maintain( register Node * cur )
{
    if( cur -> left -> size > cur -> right -> size * ratio )
        cur -> right = merge( cur -> left -> right , cur -> right ) , st[ --cnt ] = cur -> left , cur -> left = cur -> left -> left;
    else if( cur -> right -> size > cur -> left -> size * ratio )
        cur -> left = merge( cur -> left , cur -> right -> left ) , st[ --cnt ] = cur -> right , cur -> right = cur -> right -> right;
}

int rank( int x , Node * cur )
{
    if( cur -> size == 1 ) return x > cur -> value;
    return x > cur -> left -> value ? rank( x , cur -> right ) + cur -> left -> size : rank( x , cur -> left );
}

void insert( int x , Node * cur )
{
    if( cur -> size == 1 )
        cur -> left = new_Node( 1 , min( cur -> value , x ) , null , null ) ,
        cur -> right = new_Node( 1 , max( cur -> value , x ) , null , null );
    else insert( x , x > cur -> left -> value ? cur -> right : cur -> left ) , maintain( cur );
    update( cur );
}

inline void insert( int x , int y )
{
    for( register int i = x ; i <= m ; i += lowbit( i ) )
        insert( y , root[i] );
}

inline int find( int x , int y )
{
    int ans = 0;
    for( register int i = x ; i ; i -= lowbit( i ) )
        ans += rank( y + 1 , root[i] );
    return ans;
}

```

Expansion

- 树状数组套平衡树在绝大多数场合下都够用了
- 其他常见的还有树状数组套Trie，线段树套平衡树，可以类比树状数组套平衡树的实现方法来实现，即将内部的树看做一个和外部树独立的数据结构

Comparison

- 对比一下几种常见的树套树的优缺点

树状数组套树

- 优势：好写，常数
- 劣势：只能维护支持减法的信息，如果不能满足减法则基本上不能使用

树状数组套树

- 套平衡树：时间复杂度 $O(\log n^2)$ ，空间复杂度 $O(n \log n)$
- 优势：空间
- 劣势：平衡树没有简单的可以在多个平衡树上二分的方法，区间kth这种询问会多一个log（其实可以不多log的，但这个多树二分的科技常数比较大，而且较为复杂，在OI界应该算很不普及吧）

树状数组套树

- 套Trie：时间复杂度 $O(\log n \log v)$ ，空间复杂度 $O(n \log n \log v)$
- 优势：可以简单地在多个Trie上二分
- 劣势：空间（可以通过特殊的技巧达到 $O(n \log n)$ 的空间复杂度）

线段树套树

- 套平衡树：时间复杂度 $O(\log n^2)$ ，空间复杂度 $O(n \log n)$
- 套Trie：时间复杂度 $O(\log n \log v)$ ，空间复杂度 $O(n \log n \log v)$
- 优点：可以维护不支持减法的信息
- 缺点：相对难写，慢，空间大

平衡树套树

- 套平衡树：时间复杂度 $O(\log n^2)$ ，空间复杂度 $O(n \log n)$
- 套Trie：时间复杂度 $O(\log n \log v)$ ，空间复杂度 $O(n \log n \log v)$
- 优点：可以在线支持第一维插入的问题
- 缺点：挺难写，更慢，常数更大
- 注：外层树用clj定义的“重量平衡”的树
- 如treap，替罪羊树， $BB[\alpha]$ Tree，WBLT等

树套OVT

- 就是内部套一个排序后的vector
- Instead of 平衡树
- 优点：好写，空间
- 缺点：复杂度
- 还是最好别用这种奇怪的东西吧

B树套树

- 据说很快

Luogu3380 二逼平衡树

- 您需要写一种数据结构（可参考题目标题），来维护一个有序数列，其中需要提供以下操作：
- 查询k在区间内的排名
- 查询区间内排名为k的值
- 修改某一位值上的数值
- 查询k在区间内的前驱
- 查询k在区间内的后继
- $n, m \leq 5e4$

Analysis

- 平衡树中将x改为y，可以通过插入y之后删除x来实现（注意有的写法空树会导致RE，所以建议先插入后删除）
- 于是单点修改可以通过单点插入，单点删除实现

Analysis

- 查询kth可以通过二分答案后查询rank实现
- 也就是说每次二分一个答案mid, 然后查询区间中小于mid的数来确定继续二分下去的方向
- 查询前驱后继也可以用类似的方法实现

Solution1

- 发现本质就是个带单点修改的区间kth
- 可以用线段树套平衡树维护!
- 时间 $O(\log n^3)$

Solution2

- 发现区间kth可以在多个Trie上一起二分来维护
- 可以用线段树套Trie维护!
- 时间 $O(\log^2 n)$

Solution3

- 发现区间kth可以支持减法
- 可以用树状数组套Trie（权值线段树）维护！
- 时间 $O(\log n^2)$ ，很好写，常数也小

Solution4

- 通过读论文可以发现可以用动态划分树维护!
- 时间 $O((\log n / \log \log n)^2)$, 理论优越
- 不过 Not practical

Note

- 这个不断优化的过程和做这种偏序题的我的思路差不多
- 先给出一个任意poly log (比如 $\log^3, \log^4 \dots$) 复杂度的通用化的解法, 然后不断根据题目特性特殊化从而降低复杂度

四分树， 八分树

- 裸写四分树最坏的复杂度是 $O(n)$ 的
- 可以通过一些方式进行优化， 不过这里就不介绍了

高维分治

- 对于d维的情况
- 时间复杂度 $O(\log n^d)$, 空间复杂度 $O(n)$
- 优点: 空间, 常数
- 缺点: 对于有的问题会很难写, 强制在线见祖宗

K-D Tree

- 对于d维的情况
- 时间复杂度 $O(n^{1-1/d} + \log n)$, 空间复杂度 $O(n)$
- 优点: 空间, 可以支持高维空间打标记
- 缺点: 复杂度, 常数 (在不能剪枝的情况下)

如何做偏序类的题呢

- 和上面说的一样
- 反正我是先推出一个poly log的数据结构做法
- 然后想办法去log，到一个可以接受的复杂度

如何去log

- 1.降维
- 2.题目有特殊限制
- 3.在数据结构上进行二分

降维

- 可持久化
- 离线

可持久化

- 就拿静态区间kth这个问题来说
- 刚才讲过
- 显然可以用一个二维数据结构——树套树来维护
- 我决定用线段树套平衡树!
- 好!
- 现在我有了一个单次查询 $O(\log n^3)$ 的优（抠）秀（脚）做法

可持久化

- 然后可以发现
- kth是支持在数据结构上二分的信息
- 换成线段树套Trie
- 得到了一个单次查询 $O(\log n^2)$ 的优（抠）秀（脚）做法

可持久化

- 然后可以发现，这个问题不支持修改，所以没有必要用树套树，可以换用可持久化Trie（主席树）来解决
- 时间复杂度 $O(\log n)$ 单次
- 空间复杂度 $O(n \log n)$ 总

Technology

- 可以查论文，复杂度还可以优化

可持久化

- 换个麻烦点的问题：

Luogu4396 [AHOI2013]作业

- 查询区间 $[l,r]$ 内, 值域在 $[a,b]$ 内, 有多少不同的数

如何维护区间中不同的数个数

- 升一维，记录每个数上一次出现的位置
- （好像见过的所有维护区间中不同数个数的poly log题都是用的这个东西，无一例外）

分析这个问题

- 查询区间 $[l,r]$ 内，值域在 $[a,b]$ 内，有多少不同的数

第一维

第二维

第三维

- 所以这个题就是个裸的三维偏序维护问题

三维？

- 那岂不是 $O(\log n^3)$ 时间单次？
- 发现这个问题其实还是一个静态问题，所以可以通过可持久化降维
- 对于二维问题用可持久化线段树可以维护
- 对于三维问题用可持久化树套树可以维护

喵

- 可持久化线段树：拿一维来可持久化，维护这一维前缀的Trie（权值线段树），查询一个前缀中小于 x 的数个数，支持减法从而可以查询区间中小于 x 的数个数
- 可持久化树套树：拿一维来可持久化，维护这一维前缀的树套树，查询一个前缀中第一维小于 x ，第二维小于 y 的数个数支持减法从而可以查询区间中第一维小于 x ，第二维小于 y 的数个数

可持久化

- 所以可持久化就是在静态问题上可以降低一维

离线

- 1.分治代替可持久化
- 2.其他离线降维方法

分治

- 即用分治代替可持久化从而降维
- 一般被称作“整体二分”，“cdq分治”

Luogu4054 [JSOI2009]计数问题

- 一个 $n*m$ 的方格，初始时每个格子有一个整数权值。接下来每次有2种操作：
 - 改变一个格子的权值；
 - 求一个子矩阵中某种特定权值出现的个数。
-
- $n, m \leq 300$, 值 ≤ 100 , $q \leq 2e5$

Solution

- 翻译：有 $n*m$ 个点，每个点有三个权值 x_i , y_i , z_i
- 1. 把一个点的 z_i 进行修改
- 2. 查询有多少点满足
- $l1 \leq x_i \leq r1$
- $l2 \leq y_i \leq r2$
- $l3 \leq z_i \leq r3$

Solution

- 可以发现，这个是一个三维的偏序维护问题
- 有一个 $O(\log n^3)$ 单次的方法

Solution

- 可以发现 x_i , y_i , z_i 只有300, 所以可以用三维树状数组维护
- 然后发现 z_i 这一维只查询 $[z_i=13]$ 的所有点
- 所以可以通过开100个二维树状数组来去掉 z_i 这一维
- 于是就变成了一个单点修改, 矩形和的题了
- 时间复杂度 $O(\log n^2)$ 单次

Luogu3332 [ZJOI2013]K大数查询

- 1.区间 $[l,r]$ 插入 x
- （这里的插入指的是在每个位置上面放上一个新的数，不是每个数旁边放一个，可以看作每个位置是一个vector，在区间push_back了一下 x ）
- 2.区间kth

Solution

- 翻译：有 n 个点，每个点有权值 a_i, b_i
- 1. 加入 $(l_1, x), (l_1 + 1, x) \cdots (r_1, x)$ 这些点
- 2. 求满足 $l_1 \leq a_i \leq r_1$ 的所有点中 b_i 的 k th

Solution

- 怎么做呢
- 按照套路我们先建一个树套树
- 但是发现一个问题：区间插入的操作无法高效地在外层树上实现，因为无法打标记

Solution

- 发现区间插入这个操作不好维护，可以考虑把位置和权值两维交换一下
- 于是从加入 $(l1, x), (l1+1, x) \cdots (r1, x)$ 这些点
- 变为了在 x 位置加入 $[l1, r1]$ 这些点
- 从一个树套树的外层树区间修改，内层树单点修改
- 变为了一个树套树外层树单点修改，内层树区间修改

Solution

- 现在有了一个时间复杂度 $O(\log n^3)$ 单次的做法:
- 每次二分答案一个log
- 树套树两个log

Solution

- 区间kth是可以在多个数据结构上一起二分来实现的，在外层二分答案，内层check实质上浪费了一个log
- 于是可以做到时间复杂度 $O(\log n^2)$ ，在外层的线段树（或者树状数组）上二分即可
- 当然，这个题不强制在线，也可以通过各种分治来做

Notice

- 其实外部维护区间也是可以的，因为偏序问题基本上都是对称的，按什么顺序维护都是可以在同样的复杂度维护的
- 但是这个涉及到一个好不好写的问题…所以如果发现直接做比较麻烦可以试着换换偏序维护的顺序

动态逆序对

- Luogu3759 [TJOI2017]不勤劳的图书管理员
- Luogu3157 [CQOI2011]动态逆序对
- 即带修改，维护全局逆序对对数

Solution

- 翻译：有 n 个点，每个点两个属性 a_i, b_i
- 每次修改一个点的 b_i ，维护有多少点对 (i, j) 满足 $[a_i > a_j \&\& b_i > b_j]$

Solution

- 考虑每次修改的时候，对答案产生的贡献
- 假设原来是 (a_i, b_i) 变为 (a_i, b_i')
- 发现只对和被修改点有关的点对造成影响
- 贡献为减去所有和 (a_i, b_i) 有贡献的，加上所有和 (a_i, b_i') 有贡献的点对个数

Solution

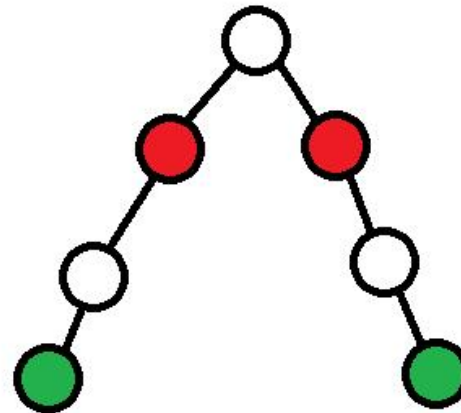
- 减去：第一维 $>a_i$ ，第二维 $>b_i$ 的，第一维 $<a_i$ ，第二维 $<b_i$ 的
- 加上：第一维 $>a_i$ ，第二维 $>b_i'$ 的，第一维 $<a_i$ ，第二维 $<b_i'$ 的
- 加上的个数和减去的个数都是一个矩形数点
- 于是用树套树可以轻松维护了
- $O(\log^2 n)$

Luogu3242 [HNOI2015]接水果

- 树，先给了一些路径，每个路径有个权值
- 然后每次查询给一个路径，求这个路径中的所有子路径里面权值kth是多少

转化

- 定义 i 点在树的DFS序的开始时间戳为 li ，结束时间戳为 ri
- 如果 $lx \leq ly \leq ry \leq rx$ 则 y 为 x 祖先
- 于是一个路径 (x,y) 的所有子路径 (u,v) 可以被定义为满足 $lx \leq lu \leq rx, ly \leq lv \leq ry$ 的所有点
- （当然还有 x 是 y 祖先的这种情况，不过这个这个差不多，就不详细说了）



转化

- 这种路径包含问题，一般都是把其左右端点的DFS序当作两个属性 a_i 和 b_i 来处理
- 求出原树的DFS序，即最开始给的路径可以被定义为 (a_i, b_i, c_i)
- a_i :第一个端点的DFS序
- b_i :第二个端点的DFS序
- c_i :权值

转化

- 即开始给出 n 个矩形，每个矩形里面插入一个数
- 每次查询单点的 k th

Solution

- 扫描线扫一维，变成区间插入，区间删除，单点kth，就转化为那个k大数查询问题了
- 这里强制在线，所以我们使用可持久化树套树来维护

Luogu 4690 [Ynoi2016]镜中的昆虫

- 维护一个长为 n 的序列 $a[i]$, 有 m 次操作。
- 1.将区间 $[l,r]$ 的值修改为 x 。
- 2.询问区间 $[l,r]$ 出现了多少种不同的数。
- 也就是说同一个数出现多次只算一个。

Solution

- 这个是个裸的区间染色段数均摊
- 考虑维护每个点与其颜色相同的前驱
- 用平衡树维护颜色连续段，然后考虑一个颜色连续段里面除了第一个点以外，后面的点 i 的前驱都是 $i-1$
- 用树套树/分治维护，于是这个颜色段的变化可以 $O(\log^2 n)$ 维护
- 总复杂度 $O((n+m)\log^2 n)$

- 注意常数...
- 小心细节...

Bzoj 3489

- 给出一个长度为 n 的序列，给出 M 个询问：在 $[l,r]$ 之间找到一个在这个区间里只出现过一次的数，并且要求找的这个数尽可能大。如果找不到这样的数，则直接输出0。

Solution

- 还是通过前驱后继来算出每个位置的贡献范围
- 设 i 的前驱为 p_i ，后继为 q_i ，把询问看成点 (L,R) ，有贡献的 i 满足 $L \in (p_i, i]$ 且 $R \in [i, q_i)$ ，询问的就是覆盖这个点的矩形的最大值。

Solution

- 那么可以用可持久化树套堆，插入矩形时一维可持久化，一维区间插入，用堆维护最大值。
- 注意这里的“可持久化堆”只需要查询历史，因此只需要把最大值记下来就好了。
- $O(n \log^2 n + m \log n)$

Luogu3688 [ZJOI2017]树状数组

- 题目过长，大家去洛谷上看吧

Solution

- 推一下性质可以发现可怜求的是后缀和
- 设数列为 A , 那么可怜求的就是 $A[l-1]$ 到 $A[r-1]$ 的和（即 $l-1$ 的后缀减 r 的后缀）,而答案为 $A[l]$ 到 $A[r]$ 的和
- 这两种答案都包含 $A[l]$ 到 $A[r-1]$ 的和, 因此只需判断 $A[l-1]$ 与 $A[r]$ 相等的概率就行了

Solution

- 考虑记下每次修改的影响，假设已知左端点 a 和右端点 b ，那么对于某一次修改区间 $[l, r]$ ，则只有当 $a \in [l, r]$ 或 $b \in [l, r]$ 时才有影响

Solution

- 设 p 为任选区间内一个数的概率，这里分三种情况讨论：
- $a \in [1, l-1], b \in [l, r]$ 时，有 $1-p$ 的概率不影响
- $a \in [l, r], b \in [l, r]$ 时，有 $1-2p$ 的概率不影响
- $a \in [l, r], b \in [r+1, n]$ 时，有 $1-p$ 的概率不影响
- 那么只要把所有的影响都合并起来就行了，设当前相同概率为 p ，当前修改不影响的概率 q ，则相同概率更新为 $p*q+(1-p)*(1-q)$

Solution

- 考虑用二维点对 (x,y) 表示 A_x 和 A_y 相等的概率
- 可以发现每次修改就是 $O(1)$ 次矩形乘一个数
- 查询就是查询单点值
- 所以用一个树套树，矩形打永久化标记，查询单点信息就行了
- $O(m \log^2 n)$

Summarization

- 省选里面的偏序类题一般都很裸，稍微推一推就可以转换成矩形操作问题
- 有几个常见的转换：
- 链 $\rightarrow$ 两端的DFS序构成的矩形
- x 子树中距离 $x \leq k$ 的点 $\rightarrow$ DFS序一维，深度一维所构成的矩形
- 区间中不同数个数 $\rightarrow$ 每个点记下前驱构成的二维点的矩形
-

一些其他的嵌套结构的问题

- 下面的题目和树套树关系不大，不过也是数据结构之间的嵌套

Luogu5073 [Ynoi2015]世界で一番幸せな女の子

- 维护序列支持:
- 1.全局加
- 2.区间最大子段和

Solution1

- 由于可以加负数
- 设 $pre[x]$ 为 x 时刻全局加的值
- 按照 pre 从小到大处理
- 这样就转换为了只加正数

Solution1

- 考虑分块
- 每块维护一下块内长为 $1 \cdots \sqrt{n}$ 的最大前缀，最大后缀，最大子段和
- 如果一个整块加了，那么这个整块块内的最大子段长度肯定是不降的

Solution1

- 每次查询的时候按照当前的全局加标记确定出每个块内的最大子段和
- 然后把跨块的询问用左边块的最大后缀和右边块的最大前缀拼起来
- 即可
- 总复杂度 $O(m\sqrt{n} + n\sqrt{n})$

Solution2

- 这个根号太屑了，考虑poly log

Solution2

- 线段树维护：
- 每个节点的左，右，内部最大子段和的凸函数
- 凸函数是一个区间的半平面交，每个下标 x 对应的意义是这个节点被整体加了 x 后，该节点的左，右，内部最大子段和的值是多少
- 设左最大子段和是 pre ，右最大子段和是 suf ，内部最大子段和是 mid

Solution2

- 然后建树的时候可以发现
- $\text{cur} \rightarrow \text{pre}[i] = \max(\text{cur} \rightarrow \text{left} \rightarrow \text{pre}[i], \text{cur} \rightarrow \text{right} \rightarrow \text{pre}[i] + \text{cur} \rightarrow \text{left} \rightarrow \text{sum});$
- $\text{cur} \rightarrow \text{suf}[i] = \max(\text{cur} \rightarrow \text{left} \rightarrow \text{suf}[i] + \text{cur} \rightarrow \text{right} \rightarrow \text{sum}, \text{cur} \rightarrow \text{right} \rightarrow \text{suf}[i]);$
- $\text{cur} \rightarrow \text{mid}[i] = \max(\text{cur} \rightarrow \text{left} \rightarrow \text{mid}[i], \text{cur} \rightarrow \text{right} \rightarrow \text{mid}[i], \text{cur} \rightarrow \text{left} \rightarrow \text{suf}[i] + \text{cur} \rightarrow \text{right} \rightarrow \text{pre}[i]);$
- 这个可以类比单点修改区间最大子段和理解

Solution2

- 我们来看看这个需要支持什么操作
- 需要支持：
 - 1. 把一个凸函数加上一个常数
 - 2. 求一个凸函数 a ，满足 $a[i] = \max(b[i], c[i])$ ，其中 b, c 为凸函数
 - 3. 求一个凸函数 a ，满足 $a[i] = b[i] + c[i]$ ，其中 b, c 为凸函数

Solution2

- 可以发现这三个性质都满足
- 而且可以通过归并两个儿子的凸函数来得到cur的凸函数
- 于是预处理复杂度 $O(n \log n)$
- 每次查询的时候直接在线段树上查询区间即可
- 总复杂度 $O((n+m) \log n)$

Luogu2824 [HEOI2016]排序

- 给出一个1到n的全排列，现在对这个全排列序列进行m次局部排序，排序分为两种：
- 1:(0,l,r)表示将区间[l,r]的数字升序排序
- 2:(1,l,r)表示将区间[l,r]的数字降序排序
- 最后询问第q位置上的数字。

Problem

- 区间排序好像不是很好维护？

Solution1

- 既然只是最后要求查询一个位置的值，我们可以考虑二分答案
- 设二分的答案为ans
- 则把所有小于ans的设为0，大于ans的设为1
- 区间升序排序就是把所有0移到左边，1移到右边
- 区间降序排序就是把所有1移到左边，0移到右边

Solution1

- 最后看查询的位置上是0还是1
- 就知道该往那边二分了~
- 总复杂度 $O(n \log^2 n)$

Luogu5612 [Ynoi2013]Ynoi

- 1.区间异或 x
 - 2.区间排序
 - 3.区间异或和
-
- 空间32MB

Solution

- 这道题由于是多次询问所以不能用前面题的做法了

Solution

- 这个区间排序有类似于颜色段均摊的性质
- 考虑使用平衡树维护每个被排序的段，每个段内开一个数据结构维护段内排序后的信息
- 每次区间排序就是把一些段合并起来，并且把两边的段进行分裂

Solution

- 先只考虑区间排序
- 使用trie来维护每个段的信息
- 这里考虑“线段树合并”的势能，每次分裂只会增加 $O(w)$ 的势能，所以复杂度是对的

Solution

- 区间xor可以在外层平衡树上打标记来实现
- 需要处理外层平衡树上的标记传到内层trie上的影响
- Trie上可以同时维护排序标记和xor标记来实现，同时需要维护trie的区间xor和
- 总时间复杂度 $O((n+m)\log n)$ ，空间复杂度 $O(n)$

Thanks for listening